

B.TECH. PROJECT ON

WEARABLE SMART SYSTEM FOR

VISUALLY IMPAIRED PEOPLE

Submitted by
Manas Pratim Das (2022UEC2697)
Md. Haider Imam (2022UEC2708)
Sandeep Poddar (2022UEC2694)
Sourabh (2022UEC2704)

Under the Guidance of
Dr. Ritu Raj Singh

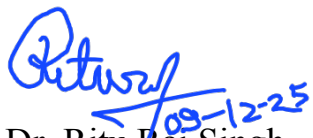
Project-I in partial fulfillment of requirement for the award of
B.Tech. in
Electronics & Communication Engineering



Department of Electronics & Communication Engineering
NETAJI SUBHAS UNIVERSITY OF TECHNOLOGY
NEW DELHI-110078
December- 2025

CERTIFICATE

Certified that Manas Pratim Das (2022UEC2697), Md. Haider Imam (2022UEC2708), Sandeep Poddar (2022UEC2694) and Sourabh (2022UEC2704) have carried out their project work presented in this project entitled “WEARABLE SMART SYSTEM FOR VISUALLY IMPAIRED PEOPLE” for the award of Bachelor of Technology, Department of Electronics and Communication, Netaji Subhas University of Technology, New Delhi, under my/our supervision. The project embodies results of original work, and studies are carried out by the student himself/herself and the contents of the project do not form the basis for the award of any other degree to the candidate or to anybody else from this or any other University/Institution.



Dr. Ritu Raj Singh

Department of Electronics & Communication Engineering
Netaji Subhas University of Technology

Date:

ACKNOWLEDGEMENT

We would like to express my gratitude and appreciation to all those who make it possible to complete this project. Special thanks to our project supervisor Dr. Ritu Raj Singh whose help, stimulating suggestions and encouragement helped us project work. We also sincerely thank our colleagues for the time spent proofreading and correcting our mistakes.

Manas Pratim Das
(2022UEC2697)

Md. Haider Imam
(2022UEC2708)

Sandeep Poddar
(2022UEC2694)

Sourabh
(2022UEC2704)

PLAGIARISM REPORT



Page 2 of 21 - Integrity Overview

Submission ID trn:oid::14362:504749774

17% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **60 Not Cited or Quoted 13%**
Matches with neither in-text citation nor quotation marks
-  **9 Missing Quotations 2%**
Matches that are still very similar to source material
-  **5 Missing Citation 2%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 12%  Internet sources
- 10%  Publications
- 9%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

ABSTRACT

Visually impaired people face significant challenges in navigating and interacting with their environments. Currently, there is no robust, reliable, and affordable system that adequately addresses these issues. Such a system can be divided into distinct stages: data collection, data storage and processing, object detection, and user feedback. This paper presents a comprehensive literature review of the various technologies and methodologies employed across these workflow stages. By integrating these technologies, a practical and widely accessible assistive system can be developed for visually impaired individuals. However, several limitations remain, including the need for compact, good processing power, and low-heat-generating hardware; limited datasets and object classes focused primarily on indoor environments; the inability to run complex models directly on microcontrollers, requiring separate servers; and a lack of effective feedback mechanisms to the user.

CONTENT

Title	Page No.
Certificate	ii
Acknowledgement	iii
Plagiarism Report	iv
Abstract	v
List of Tables	viii
List of Figures	ix
List of Symbols and Abbreviations	x
CHAPTER 1: INTRODUCTION	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Objectives	1
CHAPTER 2: LITERATURE REVIEW	3
2.1 Traditional Detection Used (Ultrasonic Sensors)	3
2.2 Evolution of Object Detection Model	4
2.2.1 Early Architecture and Two-stage Detectors	4
2.2.2 YOLO Revolution (V3 to V7)	4
2.2.3 Recent developments (V8 to V11)	4
2.2.4 Why YOLOv12?	5
2.3 IoT in Assistive Technology	6
2.3.1 ESP32-CAM	6
CHAPTER 3: METHODOLOGY	8
3.1 System Architecture and Hardware Design (Microservices-Based Architecture)	8
3.1.1 Edge Device (ESP32-CAM) – Image Acquisition Unit	8
3.1.2 Computation Core (YOLOv12n)	9

Title	Page No.
3.1.3 WebSocket Room (Node.js Backend)	9
3.2 Software Implementation	10
3.2.1 Video Acquisition Module (The Edge Layer)	10
3.2.2 Object Detection and Intelligence Engine (The AI Microservice)	10
3.2.3 Real-Time Orchestration & Communication (The Backend Microservice)	11
3.2.4 Mobile Client Application (The Frontend Layer)	12
CHAPTER 4: RESULTS AND DISCUSSION	14
4.1 Task and Achievement	14
4.2 Gantt Chart	15
4.3 Scope for Future Work	15
REFERENCES	16

LIST OF TABLES

Table 2.1: Comparison of the various YOLO models	5
--	---

LIST OF FIGURES

Figure 2.1: Architecture of YOLOv12	6
Figure 3.1: Circuit Diagram	8
Figure 4.1: System Architecture	14
Figure 4.2: Prototype Image	14
Figure 4.3: Gantt Chart	15

LIST OF ABBREVIATIONS

ABF	Adaptive Bilateral Filter
ADC	Analog-to-Digital Converter
AI	Artificial Intelligence
API	Application Programming Interface
CCTV	Closed-Circuit Television
CIoU	Complete Intersection over Union
DAC	Digital-to-Analog Converter
EATBP-ODHOA	Enhanced Assistive Technology for Blind People through Object Detection Using a Hiking Optimization Algorithm
E-ELAN	Extended Efficient Layer Aggregation Network
ETA	Electronic Travel Authorization
FLOPs	Floating Point Operations
GPIO	General Purpose Input/Output
HTTP	Hypertext Transfer Protocol
IoT	Internet of Things
IP	Internet Protocol
JSON	JavaScript Object Notation
LiPo	Lithium Polymer
mAP	Mean Average Precision
MJPEG	Motion Joint Photographic Experts Group
NMS	Non-Maximum Suppression
ODSDP-ADLMSSO	Object Detection System for Disabled People Using Advanced Deep Learning Models and Sparrow Search Optimization

PGI	Programmable Gradient Information
PQC	Post-Quantum Cryptography
QVGA	Quarter Video Graphics Array
R-CNN	Region-based Convolutional Neural Network
REST	Representational State Transfer
RPN	Region Proposal Network
SCODL-ODC	Sine Cosine Optimization with Deep Learning driven Object Detector
VGA	Video Graphics Array
Wi-Fi	Wireless Fidelity
YOLO	You Look Only Once

CHAPTER 1

INTRODUCTION

1.1 Background

Visual impairment is a highly impactful disability, affecting millions of people worldwide. According to the World Health Organization, nearly 2.2 billion individuals suffer from near or farsightedness, with approximately 300 million experiencing moderate to severe visual impairment[1]. This greatly complicates daily tasks and routines. While traditional assistive devices like canes provide tactile information about ground-level obstacles, they lack the ability to convey semantic information about the environment. Consequently, users cannot distinguish between objects such as chairs and people. Some advanced assistive technologies, such as Meta Glasses or Apple Vision, exist; however, they are often not accessible to the general population due to cost considerations. With recent developments in Internet of Things (IoT) devices and low-power object detection models, there is an opportunity to bridge this gap. This project proposes a smart wearable device that functions as an 'eye' for the user, offering auditory feedback to enhance environmental awareness.

1.2 Problem Statement

Current electronic travel authorization (ETA) systems often depend on ultrasonic sensors that produce basic beeping signals. However, these systems tend to underperform in crowded environments. There is a notable gap in affordable, low-power, real-time navigation solutions capable of accurately classifying objects and relaying information to users with minimal latency.

1.3 Objectives

The main objectives of this project are to develop a smart wearable system that can provide visually impaired users with real time information about their surrounding environments using an integrated camera module, implementing object detection, efficient communication protocols, and audio-based feedback system.

To select a lightweight and user-friendly camera module that can continuously capture the user's surroundings (video feed) with minimal power consumption and also comfortable on wearing this. For real-time object detection, we will utilize YOLOv12[2], a state-of-the-art, deep-learning-based model known for its low computational requirements.

To establish a WebSocket protocol for communication between the ESP32-CAM[3] and the server. This is essential for achieving fast, low-latency, and bidirectional communication, which in turn minimizes frame transmission and processing delays. Utilizing WebSockets[4] reduces the load on the server by eliminating continuous API update requests, thereby mitigating server bottlenecks and the communication delays

associated with them. For feedback system, transmitting information about detected objects to the user in form of audio messages through pre-recorded audio.

CHAPTER 2

LITERATURE REVIEW

2.1 Traditional Detection Used (Ultrasonic Sensors)

Traditional method of detecting objects, i.e. by using an ultrasonic sensor. An ultrasonic sensor is a device which uses ultrasonic sound transmitter and ultrasonic sound receiver. This ultrasonic sensor emits ultrasonic waves. The transmitter-receiver pair together provides the signal using which distance to the object can be calculated using mathematical computations. This signal is sensed by ESP-32 and distance is computed.

In this paper[5], the author implemented using three ultrasonic sensors (S_F , S_R , S_L) to detect more accurate objects. These sensors emit 40Khz ultrasonic waves and determine the time taken by these waves to return after encountering the obstacles. Using three sensors, objects can be detected in front, right and left side which improves the accuracy of the device. The front sensor (S_F) monitors objects in the front, while the right sensor (S_R) is responsible for tracking objects on the right side, and the left sensor (S_L) keeps tabs on the left side. When the distance in front (F_D) falls below the safe distance (S_D) of 200cm, the ESP-32 triggers the audio amplifier, producing a distinctive low-frequency (75Hz) audio signal, accompanied by the vocal warning "Obstacle in Front." The ESP-32 also assesses the distances measured by S_L and S_R . If the left distance (L_D) is less than S_D , it commands the audio amplifier to articulate 'Turn right.' Conversely, if the right distance (R_D) is less than S_D , it directs the audio amplifier to announce "Turn left".

In this paper[6], the author modified the ultrasonic sensor (HC-SR04) in the infrastructure module. The ultrasonic sensor in the infrastructure module does not have a transmitter unit. The author achieved by de-soldering the transmitter pins and shorting the exposed pins. Using this modification, the receiver can listen any ultrasonic sound with specific pattern. The Infrastructure Module has a dedicated microcontroller. The microcontroller is powered by a 3.7V LiPo battery. However, the ultrasonic sensor requires 5V input. The author used a boost converter to step up the voltage. Once a client establishes connection with the Infrastructure Module, it continuously polls the ultrasonic sensor to determine if the client module is directly below itself. Additionally, whenever the client is detected, the Infrastructure Module notifies the client that it is directly below that Infrastructure Module.

In this paper[7], the author implemented the obstacle detection system using ultrasonic sensor which measures both distance and echo when the object is detected. The sensor has four pins including VC, GND, Trigger (output pin) and echo (input pin). The sensor emits the waves like bat spreading the waves and listen for the echoes of nearby obstacles. It works on the principle of transmitting ultrasonic waves and receive the reflected waves. The author uses both ultrasonic sensor and buzzer which is connected to the microcontroller board to collect the data of these sensors which is particularly useful for visually impaired individuals in identifying and navigating around obstacles.

2.2 Evolution of Object Detection Model

Object detection is a crucial computational component of the proposed assistive system. The evolution of these algorithms has shifted from high-latency, two-stage detectors to efficient, single-stage detectors capable of operating on constrained hardware architectures. This section reviews the development of these models, examining their trade-offs within the context of visual impairment assistance.

2.2.1 Early Architecture and Two-stage Detectors

Initial research into deep learning-based navigation extensively utilized Region-based Convolutional Neural Networks (R-CNN). Notably, Faster R-CNN[8] emerged as a prominent architecture, employing a Region Proposal Network (RPN) combined with a secondary classifier. The loss function for Faster R-CNN[8] is given by:

$$L_{Faster\ R-CNN} = L_{cls}^{RPN} + L_{reg}^{RPN} + L_{cls}^{Det} + L_{reg}^{Det} \quad 1[8]$$

While models such as EATBP-ODHOA[8], leverage Faster R-CNN[8] to attain high precision in detecting small objects, they are hindered by substantial computational demands. Comparative studies indicate that such models can require significant processing times—up to 10.87 seconds in unoptimized environments—posing dangerous latency risks for blind users navigating dynamic settings.

2.2.2 YOLO Revolution (V3 to V7)

To mitigate the latency issues inherent in two-stage detectors, the YOLO (You Only Look Once) family was introduced, re-framing detection as a single regression problem.

- **YOLOv3 and v4:** These iterations incorporated multi-scale prediction capabilities but still had relatively high parameter counts.
- **YOLOv5:** A pivotal release employed in the SCODL-ODC[9] model, using a backbone for feature extraction and a neck component for multi-scale fusion, achieving an impressive balance between speed and accuracy. Comparative analyses demonstrate YOLOv5[10] attaining processing times as low as 2.92 seconds in optimized environments, making it far better suited for wearable applications than R-CNN.
- **YOLOv6 and v7:** YOLOv7[11], utilized in the ODSDP-ADLMSSO[12] model, introduced further improvements with the Extended Efficient Layer Aggregation Network (E-ELAN) backbone. This architecture allows the model to learn diverse features without destroying the gradient path, making it highly effective for real-time applications.

2.2.3 Recent developments (V8 to V11)

Following version 7, the evolution of YOLO has concentrated on architectural enhancements:

- **YOLOv8:** Introduced an anchor-free detection head, simplifying the training process and improving adaptability to various object shapes[13].

- **YOLOv9:** Focused on Programmable Gradient Information (PGI) to minimize information loss during deep network transmission[14].
- **YOLOv10 and v11:** Advanced efficiency by implementing Non-Maximum Suppression (NMS)-free training and improved feature extraction modules, further reducing inference latency on edge devices[15], [16].

2.2.4 Why YOLOv12?

For this project, YOLOv12[2] has been chosen as the detection platform. It embodies the latest advancements within the YOLO family, addressing specific limitations identified in earlier versions such as v5[10] and v7[11].

Table 2.1: Comparison of the various YOLO versions

Model	Size (pixel)	mAP (50-95)	Latency (ms)	Params (M)	FLOPs (B)
YOLO12n[2]	640	40.6	1.64	2.6	6.5
YOLO11n[16]	640	39.5	1.5	2.6	6.5
YOLOv10n[15]	640	38.5	1.84	2.3	6.7
YOLOv9s[14]	640	46.8	3.54	7.1	26.4
YOLOv8n[13]	640	37.3	0.99	3.2	8.7
YOLOv7tiny-SiLU[11]	640	38.7	3.49	6.2	13.8
YOLOv6n[17]	640	37.5	1.17	4.7	11.4
YOLOv5nu[10]	640	34.3	1.06	2.6	7.7

The decision is grounded in three key improvements:

1. **Attention-Centric Architecture:** Unlike YOLOv5[10] or v7[11], which primarily depend on CNN backbones, YOLOv12[2] incorporates advanced FlashAttention mechanisms. This enables the model to prioritize salient features (e.g., a vehicle approaching the user) while disregarding background noise, a crucial requirement for safety.
2. **Latency V/S Accuracy Trade-off:** Although models like Faster R-CNN[8] offer high precision, they are too slow for real-time applications. Conversely, MobileNet-based models[] are fast but often miss small obstacles. YOLOv12[2] achieves a higher Mean Average Precision (mAP) than YOLOv8[13] and v10[15], while maintaining inference speeds in the millisecond range necessary for the system's real-time sampling intervals of approximately five seconds.
3. **Small Object Detection:** Literature indicates that earlier models struggle with low-height obstacles or small hazards, such as potholes. The enhanced feature aggregation in YOLOv12[2] significantly improves performance on small objects, directly addressing a notable gap in existing assistive devices.

By employing YOLOv12[2], this system overcomes the computational limitations associated with two-stage detectors while maintaining high detection accuracy and real-time performance.

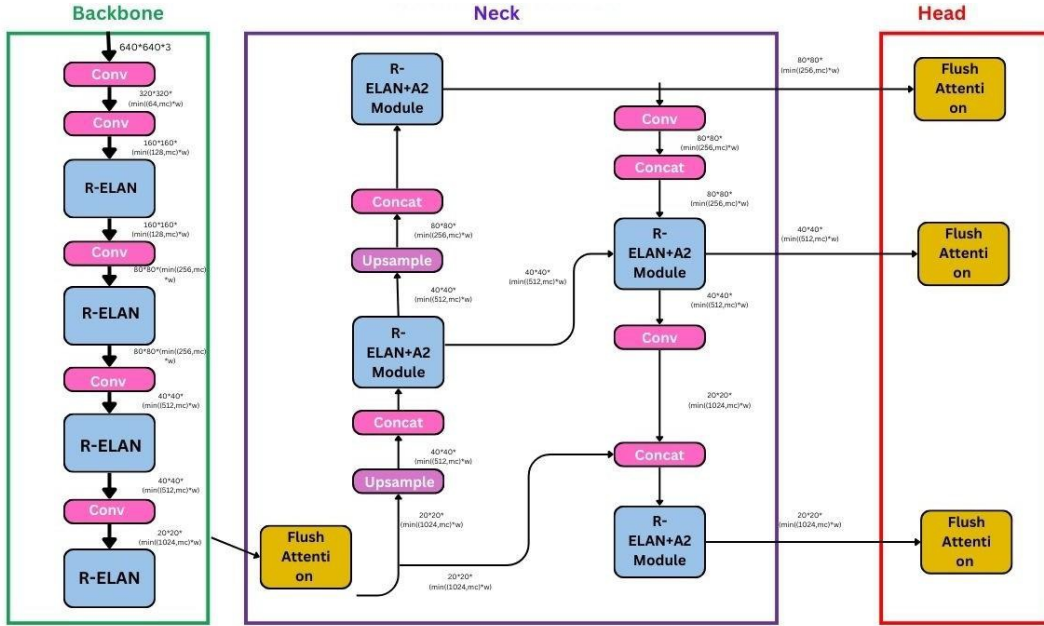


Figure 2.1: Architecture of YOLOv12[23]

2.3 IoT in Assistive Technology

IoT devices is rapidly increasing in the field of assistive technology because these devices can collect, process, and transmit real-time data. IoT-based solutions make daily life tasks safer and easier for visually impaired users.

Choosing ESP32-CAM[18] for the main project. This microcontroller was chosen because of its small size, wearable design, integrated camera and Wi-Fi, low power consumption, affordability, ease of integration with IOT architecture, and—most importantly—suitability for real-time streaming.

2.3.1 ESP32-CAM

This module is a Wi-Fi module that is equipped with a camera. This module can be used for various purposes, for example for CCTV, taking pictures and so on. Another feature is that it can detect faces and face recognition. The ESP32-CAM[18] has a very competitive small size camera module that can operate independently with a minimum system. ESP32-CAM[18] can be used widely in various IoT applications. It is suitable for smart home devices, industrial wireless control, wireless monitoring, QR wireless identification, wireless positioning system signals and other IoT applications.

The main benefit of using ESP32-CAM[3] is that it comes with a dual-core Tensilica Xtensa LX6 microprocessor (up to 240 MHz), unlike many single-core devices which makes it suitable for handling more complex tasks for real time processing. It has Wi-

Fi (802.11 b/g/n) and Bluetooth (4.2 BR/EDR and BLE) in the module, so that external modem chips are not required, thus simplifying design. It also offers various peripheral I/O pins such as 34 General Purpose Input/Output (GPIO) pins, 18 channel 12-bit SAR ADC (Analog-to-Digital Converter), 2 channel 8-bit DAC (Digital-to-Analog Converter), multiple UART, I2C, I2S, and SPI connections, Pulse Counter module and Capacitive Touch. It Supports multiple power saving modes (i.e., modem-sleep, light-sleep, deep-sleep), therefore can be used whenever battery-operated and designed to run for longer periods of time.

CHAPTER 3

METHODOLOGY

3.1 System Architecture and Hardware Design (Microservices-Based Architecture)

The wearable smart system follows a microservices-based distributed architecture. So that system ensures modularity, low latency, reliable processing, and real-time assistance for visually impaired people. The system is divided into three independent computational nodes, each responsible for a specific task: image acquisition, object detection, and communication management.

3.1.1 Edge Device (ESP32-CAM) – Image Acquisition Unit

The wearable prototype's main edge sensing module is the ESP32-CAM[3]. Its primary duties consist of video capture in real time, utilizing its onboard IP camera interface to output a raw MJPEG[19] video stream and continuously sending frames to the server via the local network. By functioning just as a camera acquisition device, this node offloads computation, guaranteeing low power consumption and a small form factor appropriate for wearable integration.

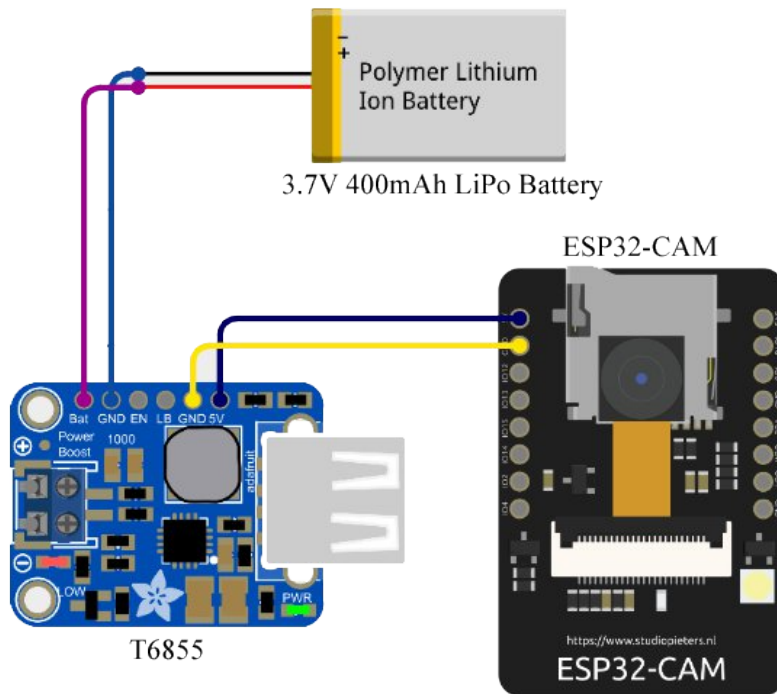


Figure 3.1: Circuit Diagram

Initially, the device is set to function as an access point so as to retrieve the Wi-Fi credentials from the mobile application. This communication is done on the private

network setup by the ESP32-CAM[18]. Once it is connected to the Wi-Fi the access point is turned off and the device establishes connection with the main backend server. After this it makes an API call to the server to save the video streaming IP address in the database.

3.1.2 Computation Core (YOLOv12n)

All high-level computational operations—including preprocessing and object detection—are executed on a Python script running locally (or remotely) on a server. Once it establishes connection with the backend server, it fetches for the IP address of the video stream from where it can capture the frames.

This node performs the task of decoding the raw frames coming from ESP32-CAM[3], normalize and resize the frames according to YOLOv12n[2] input requirements.

For object detection, the lightweight YOLOv12n[2] model was chosen due to its ideal speed-accuracy ratio and real-time assistive systems. In output, this gives both class labels of detected object and bounding box coordinated for spatial localization.

The Python server determines the detected class after each inference and sends the metadata into the room to ensure user isolation with the *sendFrame* event. The server is then responsible for appropriate processing of this data.

3.1.3 WebSocket Room (Node.js Backend)

The system's main communication mediator is the Node.js backend, which is hosted on Render.com. It is designed as a scalable and asynchronous microservice. The key functions are:

- Real-time data delivery using WebSockets
- WebSocket room management for user isolation
- Lightweight forwarding logic
- Synchronized transmission of audio and frame data

In real-time data delivery, a persistent two-way connection with the mobile device is made possible by the Node.js backend's use of WebSockets[4] rather than HTTP polling. Polling delays and repeated update requests are eliminated as a result. The server can instantly push detection results, stream metadata, and send audio cues when the channel is always open, resulting in significantly lower latency and timely environmental feedback for visually impaired users.

The Node.js backend employs a room-based WebSocket[4] architecture to guarantee privacy and proper data routing. Every wearable device or user is given a distinct room ID, and the server only sends processed frames and audio cues to that particular room. This guarantees that no two users share the same communication channel and stops any cross-talk between devices. The system can support several users at once while upholding strict data separation and dependable communication thanks to this isolated room management.

The Python server sends detection payloads, such as object labels, bounding box data, confidence scores, and pre-recorded audio clips, to the Node.js backend, which serves as a lightweight forwarding microservice. It then packages these payloads into a structured message and instantly pushes the output to the relevant WebSocket[4] room. Response times stay incredibly low because Node.js doesn't do any heavy computation; all intensive processing is done on the cloud server. In order to avoid delays between visual detection and audio guidance, the backend additionally guarantees synchronized transmission of audio cues and frame metadata in a single payload. In order to ensure user safety in a wearable assistive system, this real-time synchronization is crucial for providing precise and timely alerts like "No objects detected," "Person and car detected", or "A lot of objects detected"

3.2 Software Implementation

The software architecture of the proposed system is designed as a distributed, event-driven microservices ecosystem. Unlike monolithic assistive devices that attempt to perform all computations on-board, this system decouples **Data Acquisition (Edge)**, **Intelligence (AI Server)**, **Orchestration (Cloud Backend)**, and **Interaction (Client)**. This separation of concerns allows for the utilization of heavy state-of-the-art deep learning models (YOLOv12) without compromising the battery life or form factor of the wearable device.

The implementation is divided into four distinct operational modules.

3.2.1 Video Acquisition Module (The Edge Layer)

The primary input node of the system is the ESP32-CAM module, which operates in a passive streaming mode to conserve energy.

The ESP32 is programmed using C++ to initialize the OV2640 camera sensor. Instead of processing frames locally, it establishes a lightweight web server that broadcasts the video feed as an MJPEG (Motion JPEG) stream over a local IP address.

The resolution is capped (e.g., QVGA or VGA) to balance bandwidth usage with detection accuracy. The module strictly handles I/O operations, ensuring minimal power draw.

3.2.2 Object Detection and Intelligence Engine (The AI Microservice)

The core computational workload is offloaded to a dedicated Python-based microservice. This server acts as the system's visual cortex, responsible for interpreting the raw video stream.

A. YOLOv12 Integration

The model utilizes the YOLOv12[2] (You Only Look Once) architecture for object detection. Unlike its predecessors, YOLOv12 integrates FlashAttention mechanisms, allowing the model to focus on salient features (dynamic obstacles) while suppressing background noise.

The detection process treats the problem as a single regression task, predicting bounding boxes (B) and class probabilities (C) simultaneously. The model optimizes the following multi-part Loss Function (L_{total}):

$$L_{total} = \lambda_{box}L_{box} + \lambda_{obj}L_{obj} + \lambda_{cls}L_{cls} \quad 2[20]$$

where,

- **L_{box} (Localization Loss):** Measures the error between the predicted bounding box coordinates (x, y, w, h) and the ground truth. Modern YOLO implementations often utilize CIoU (Complete Intersection over Union) loss to account for overlap, distance, and aspect ratio consistency:

$$CIoU = 1 - IoU + \frac{\rho^2(b, b^{gt})}{c^2} + \alpha v \quad 3[21]$$

- **L_{obj} (Objectness Loss):** Penalizes the network if it detects an object where none exists (background).
- **L_{cls} (Classification Loss):** Calculates the cross-entropy error for the class predictions (e.g., distinguishing a "Chair" from a "Person").

B. Audio Generation Logic

To enable vocal feedback, the system utilizes the gTTS (Google Text-to-Speech) library to pre-generate audio assets. A Python script iterates through the 80 COCO object classes and essential system phrases (e.g., "detected," "no objects"). For each entry, the script converts the text to speech and saves it as an *.mp3* file in a dedicated assets directory. Filenames are programmatically cleaned by replacing spaces with underscores (e.g., *traffic_light.mp3*) to ensure seamless retrieval during runtime.

3.2.3 Real-Time Orchestration & Communication (The Backend Microservice)

To bridge the gap between the Python AI server and the end-user, a Node.js backend (hosted on Render.com) serves as the central orchestrator. This layer is critical for scalability and session management.

A. WebSocket Architecture

Reliance on standard HTTP (REST) protocols introduces a significant bottleneck known as "polling" latency. In this traditional request-response model, the client must repeatedly query the server to check for new data, creating unavoidable delays between data availability and data retrieval. For a safety-critical application designed for blind navigation, where milliseconds impact reaction time, this latency is unacceptable. The overhead of establishing frequent HTTP connections fails to meet the strict real-time requirements necessary for obstacle avoidance.

To overcome these latency issues, the system implements a full-duplex communication channel using WebSocket[4]. This architecture utilizes an event-driven "push" model rather than the traditional "pull" method. Instead of waiting for the client to request updates, the Node.js backend instantly emits an event to the client the moment the Python inference service completes its processing. This ensures that processed obstacle data is delivered immediately without the need for repetitive requests, guaranteeing the fastest possible feedback loop for the user.

B. Room-Based Isolation

To prevent data cross-talk (e.g., User A receiving User B's navigation cues), the backend implements a "Room" logic. The first step is to connection with the server and then the user is expected to join a specific *roomId*.

The Python sends the frame metadata by tagging it with a *roomId* to the Node.js server. Then the server routes the message strictly to the socket connection associated with that room. This method prohibits any cross communication between two users, thereby preventing any chances of fatal accidents.

C. Audio Stitching & Arraying

The backend performs the final data transformation. It receives raw audio buffers or file paths from the AI service and "stitches" them into a sequential array. This ensures that the client receives a single, coherent playable packet rather than disjointed sounds.

3.2.4 Mobile Client Application (The Frontend Layer)

The user interface is built using Flutter, selected for its high-performance rendering engine (Skia) and cross-platform capabilities.

A. Data Consumption Pipeline

The app maintains a persistent WebSocket[4] connection to the Node.js backend. The data flow within the app follows a strict state management cycle. The app subscribes to the *receiveFrame* socket event. This connection allows it to listen to any new frames sent to the room.

The incoming JSON payload containing the frame, message and audio array is parsed. Here, the audio array is the most crucial data which is used by the app to announce if any object is detected ahead and the rest is used by the Flutter for displaying the frame and message on the mobile app.

The audio buffers are pushed to a playback queue. To prevent audio overlap (which can confuse visually impaired users), the app plays the stitched audio sequentially, ensuring the previous alert finishes before the next begins.

B. Latency Management

To ensure the "Real-Time" requirement is met, the system operates on a defined sampling interval, i.e. 5 seconds. This interval is synchronized across the microservices to prevent buffer bloat and ensure that the user receives the most current environmental snapshot available.

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Task and Achievement

The main objective of this project was to develop a system which deliver real-time obstacle notifications to the user who is visually impaired. The incorporation of WebSocket[4] reduced the load on the server connecting the various technologies, which in turn helped in load balancing and reducing the latency at the interfaces. These latencies pile up when the network is being shared by multiple devices, therefore any reduction in latency helps reduce the load exponentially on the server.

Furthermore, the YOLOv12[2] model accurately detects objects in record time. This plays two crucial roles. First, is the real-time application need and the other is the reliability of the system. These two factors play a crucial role in the performance of the system.

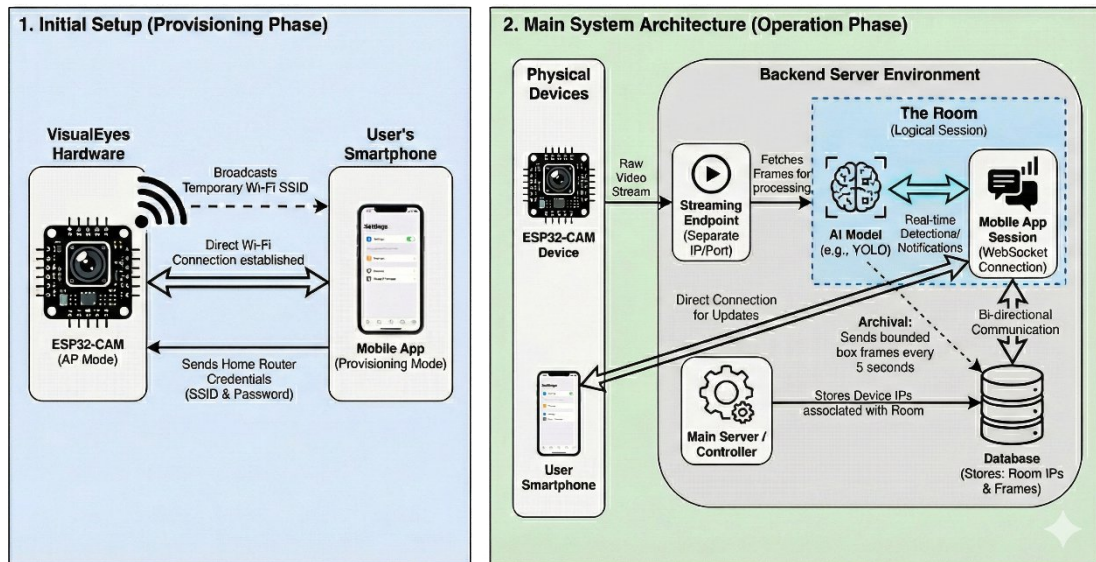


Figure 4.1: System Architecture

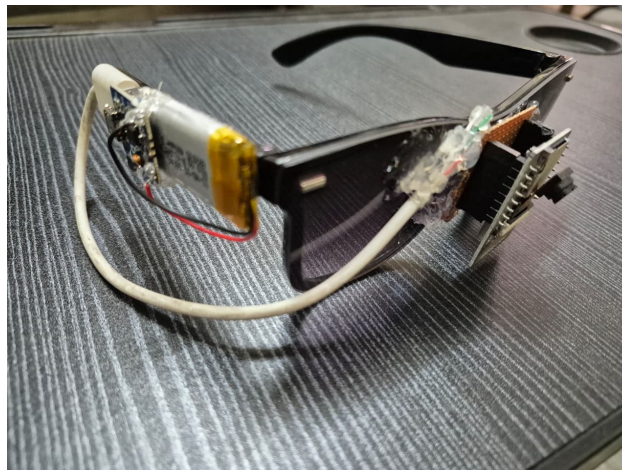


Figure 4.2: Prototype Image

Therefore, it is concluded that the system performs as intended with very high reliability and efficiency.

4.2 Gantt Chart

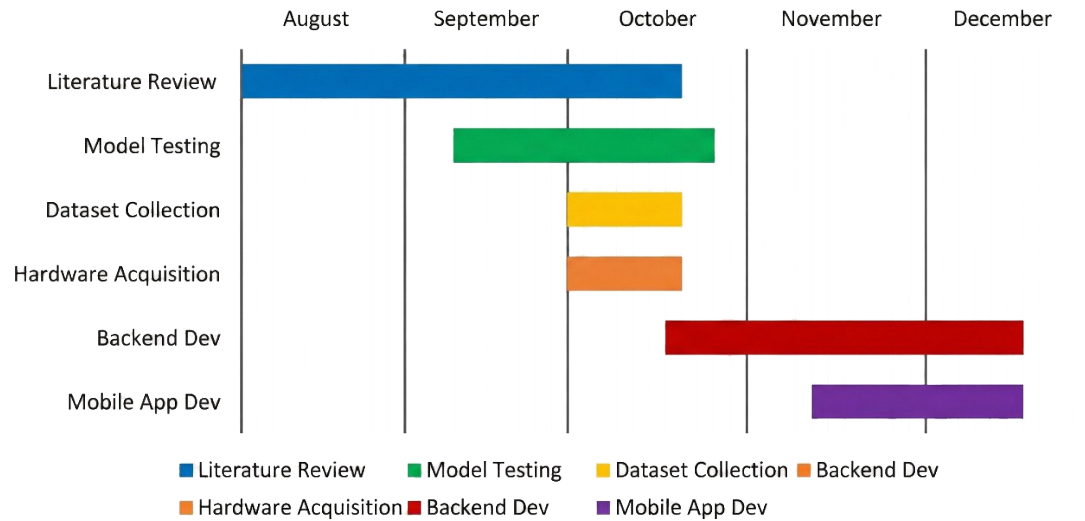


Figure 4.3: Gantt Chart

4.3 Scope for Future Work

There are still certain issues with the current system that could be fixed in later iterations. Data privacy is one of the main issues. There is always a chance that private information about the user's environment could be revealed if the data is intercepted because video frames may be stored or travel through cloud servers.

Quantum computing presents another new challenge. In the future, it might be simpler to crack current encryption techniques due to the development of strong quantum machines. The "save now, decrypt later" strategy allows attackers to store encrypted data now and wait for quantum computers to become powerful enough to decrypt it.

Future research can examine post-quantum cryptography (PQC)[22], which is intended to withstand quantum attacks, in order to mitigate these risks. By incorporating such security techniques, the system's visual data and communication can be kept safe for many years.

REFERENCES

- [1] “World Report on Vision,” 2019.
- [2] Y. Tian, Q. Ye, and D. Doermann, “YOLOv12: Attention-Centric Real-Time Object Detectors,” *arXiv preprint*, Feb. 2025.
- [3] A. Maier, A. Sharp, and Y. Vagapov, *Comparative Analysis and Practical Implementation of the ESP32 Microcontroller Module for the Internet of Things*. IEEE, 2017. doi: 10.1109/ITECHA.2017.8101926.
- [4] V. Pimentel and B. G. Nickerson, “Communicating and Displaying Real-Time Data with WebSocket,” *IEEE Internet Comput*, vol. 16, no. 4, pp. 45–53, Jul. 2012, doi: 10.1109/MIC.2012.64.
- [5] D. Pujara, R. Gautam, B. Sabuwala, and D. Shah, “Enhanced Smart Stick Design for Visually Impaired,” in *Proceedings - 2023 IEEE International Symposium on Smart Electronic Systems, iSES 2023*, Institute of Electrical and Electronics Engineers Inc., 2023, pp. 445–448. doi: 10.1109/iSES58672.2023.00100.
- [6] H. G. Kusneniwar, S. Ghosh, and S. Sen, “SonicGlass: An Obstacle Detection and Navigation System Using Smartglass-Based Ultrasonic Sensors,” in *2024 16th International Conference on COMMunication Systems and NETWORKS, COMSNETS 2024*, Institute of Electrical and Electronics Engineers Inc., 2024, pp. 603–607. doi: 10.1109/COMSNETS59351.2024.10427277.
- [7] S. Ikram, I. S. Bajwa, A. Ikram, I. D. L. T. Diez, C. E. U. Rios, and A. K. Castilla, “Obstacle Detection and Warning System for Visually Impaired Using IoT Sensors,” *IEEE Access*, vol. 13, pp. 35309–35321, 2025, doi: 10.1109/ACCESS.2025.3543299.
- [8] M. M. S. Adam, N. O. Aljehane, M. Y. Alzahrani, and S. Al Zanin, “Leveraging assistive technology for visually impaired people through optimal deep transfer learning based object detection model,” *Sci Rep*, vol. 15, no. 1, Dec. 2025, doi: 10.1038/s41598-025-14946-5.
- [9] H. Alsolai, F. N. Al-Wesabi, A. Motwakel, and S. Drar, “Sine Cosine Optimization With Deep Learning–Driven Object Detector for Visually Impaired Persons,” *Journal of Mathematics*, vol. 2025, no. 1, 2025, doi: 10.1155/jom/5071695.
- [10] G. Jocher, “Ultralytics YOLOv5,” 2020. doi: 10.5281/zenodo.3908559.
- [11] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, “YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors,” Jul. 2022.
- [12] M. Obayya, F. N. Al-Wesabi, M. Alshammeri, and H. G. Iskandar, “An intelligent optimized object detection system for disabled people using advanced deep learning models with optimization algorithm,” *Sci Rep*, vol. 15, no. 1, Dec. 2025, doi: 10.1038/s41598-025-00608-z.
- [13] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8,” 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [14] C.-Y. Wang, I.-H. Yeh, and H.-Y. M. Liao, “YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information,” Feb. 2024.
- [15] A. Wang *et al.*, “YOLOv10: Real-Time End-to-End Object Detection,” Oct. 2024.
- [16] G. Jocher and J. Qiu, “Ultralytics YOLO11,” 2024. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [17] C. Li *et al.*, “YOLOv6 v3.0: A Full-Scale Reloading,” Jan. 2023.
- [18] P. W. Rusimamto, Endryansyah, L. Anifah, R. Harimurti, and Y. Anistyasari, “Implementation of arduino pro mini and ESP32 cam for temperature monitoring on automatic thermogun IoT-based,” *Indonesian Journal of Electrical Engineering and Computer Science*, vol. 23, no. 3, pp. 1366–1375, Sep. 2021, doi: 10.11591/ijeecs.v23.i3.pp1366-1375.

- [19] B. Smith, “Fast software processing of motion JPEG video,” in *Proceedings of the second ACM international conference on Multimedia - MULTIMEDIA '94*, New York, New York, USA: ACM Press, 1994, pp. 77–88. doi: 10.1145/192593.192628.
- [20] J. Redmon, R. Girshick, S. Divvala, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” 2016.
- [21] Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye, and D. Ren, “Distance-IoU Loss: Faster and Better Learning for Bounding Box Regression,” 2016. [Online]. Available: <https://github.com/Zzh-tju/DIoU>.
- [22] K. Mansoor, M. Afzal, W. Iqbal, and Y. Abbas, “Securing the future: exploring post-quantum cryptography for authentication and user privacy in IoT devices,” *Cluster Comput*, vol. 28, no. 2, Apr. 2025, doi: 10.1007/s10586-024-04799-4.
- [23] A. Nagesh Chitari, D. Inamadar, and D. Salve, “Deep Learning-Based NIR Face Detection under Adverse Illumination with Explainable AI,” 2024. doi: 10.52783/jisem.v10i36s.6541.